

Reading Machine Health at the Edge: A Fog-to-Cloud Predictive-Maintenance Monitor for a Two-Line Plant Floor

Vikas Reddy Amanagantti
Student ID: X25178849
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x25178849@student.ncirl.ie

Abstract—A production machine rarely fails without warning. It first runs a little rougher, a little hotter, a little louder, drifts off its rated speed, or begins to draw more current than the work demands, and a maintenance team that catches those early signs can service the machine on its own terms rather than after it has stopped the line. This project builds a monitor for exactly that early warning across a scaled plant floor. Five condition signals, vibration, motor temperature, bearing acoustic level, rotation speed, and power draw, are read from ten sensor processes spread over two production lines; a fog node on the edge host windows each machine-and-signal stream, reduces it to a summary, and runs the alarm rules there, so an emerging fault is named in the window it appears rather than after a trip to a distant region. A serverless backend on Amazon SQS, AWS Lambda, and Amazon DynamoDB persists those summaries, and a plant-floor dashboard served through Amazon API Gateway turns them into a per-line, per-machine health board, with the static frontend delivered from Amazon S3. The alarm rules reflect how the signals actually fail: most worsen only as they climb, but rotation speed is faulted at both ends, because a healthy machine must stay inside a speed band rather than merely below a ceiling. Each tier is justified against the alternatives it displaces, and the system was provisioned to a real Amazon Web Services account rather than to a local emulator. An automated suite of 46 tests passes, and verification against the live endpoint confirmed both lines streaming all five signals, the alarm banner correctly naming a fault on the affected line, and the archived record count climbing as the pipeline ran.

Keywords—predictive maintenance, condition monitoring, Industry 4.0, fog computing, edge computing, Internet of Things, serverless computing, Amazon SQS, AWS Lambda, Amazon DynamoDB

I. INTRODUCTION

The economics of a plant floor turn on unplanned downtime. A machine that stops without warning halts the line it sits on, and the cost of that stoppage usually dwarfs the cost of the repair itself, which is why maintenance has moved from fixing what has broken toward reading the signs that a machine is beginning to degrade and servicing it before it fails. That shift, from reactive to predictive maintenance, is one of the defining data problems of the connected factory: instrument the machines, watch the right signals continuously, and turn the stream into a timely warning [1], [2]. The signals

themselves are well understood. Rising vibration betrays a wearing bearing, a climbing motor temperature betrays cooling or load trouble, an elevated acoustic level betrays a developing mechanical anomaly, a drift off rated rotation speed betrays a control or mechanical fault, and a rising power draw betrays friction or an electrical problem. Read together, and continuously, they say whether a machine is healthy long before it stops.

The distributed-systems question is where that reading happens. The Internet of Things makes it cheap to put sensors on every machine [3], but streaming every raw sample from every machine to a distant cloud region to be judged there is wasteful in two directions at once. It spends bandwidth continuously on readings that, taken singly, seldom change a decision, and it defers the one moment that matters, a signal crossing into its alarm range, until after a network round trip. Fog computing was proposed to close that gap by placing a compute tier between the sensors and the cloud, near enough to the machines to summarise, filter, and evaluate rules before anything travels [4], [5], and it is one expression of the broader movement of computation back toward the network edge [6], [7]. Behind that edge tier, the cloud supplies the elastic, pay-as-used model that removes the need to size a server for a peak that is idle most of the time [8].

This project builds that two-tier design for a scaled but faithful slice of a plant floor: two production lines, designated line-1 and line-2, each carrying the same five condition signals, run as ten independent sensor processes. A fog node on the edge host receives their readings, groups each line-and-signal pair into a fixed time window, reduces the window to summary statistics, evaluates the alarm rules against the summary, and forwards only the finished window and any alarm it raises. A serverless cloud pipeline persists each window, and a plant-floor dashboard folds the persisted history into a per-line board that shows each machine signal against its alarm limit and names any fault that is currently firing.

The work pursues four objectives, each shown on a running system rather than only described. The first is a sensor and fog layer that genuinely windows and evaluates five heterogeneous

condition signals at sampling and dispatch rates that are configurable rather than fixed in code. The second is a backend that scales through managed, elastic AWS primitives instead of a single always-on server. The third is a dashboard that turns stored windows into an operational reading a maintenance team could act on without inspecting raw values. The fourth, and the one that separates a working system from a plausible one, is deployment onto a real public-cloud account and verification against the live endpoint. Scope is held to these layers at a two-line scale rather than a whole factory. The remainder of this paper is organised as follows. Section II presents the architecture and the reasoning behind each choice, including the alternatives weighed and the security posture. Section III covers the implementation, the automated tests, continuous integration, the real deployment, and the evidence gathered from the running system. Section IV closes with a critical assessment of the trade-offs, the limits, and where the work could go next.

II. SYSTEM ARCHITECTURE AND DESIGN

Fig. 1 shows the deployed system end to end: ten sensor processes and the fog node sharing one edge host, a managed queue, two independently deployed functions, a key-value store, and a browser-facing bucket reached through a managed gateway.

A. Sensor and Fog Layer

Each of the ten sensor processes represents one condition signal on one line. On every sampling tick a process advances a bounded random walk: it nudges its previous value by a step drawn uniformly from a small symmetric range and clamps the result into a physically plausible band for that signal, so consecutive readings stay close together like a real transducer trace rather than independent noise. The plausible bands differ sharply by signal, and that difference is the heterogeneity the design has to carry: vibration sits in single-digit millimetres per second, temperature in the tens to low hundreds of degrees, acoustic level in the tens of decibels, rotation speed in the hundreds to thousands of revolutions a minute, and power draw in the tens of kilowatts. A process samples on one interval and dispatches on another, and the two are independent so a fast-moving quantity can be sampled often while still being shipped in economical batches; the sampling interval defaults to two seconds and the dispatch interval to ten. On each dispatch the accumulated readings are posted as a compact JSON object naming the signal, the line, and the unit and carrying the batch of values gathered since the last dispatch, which at the default rates is roughly five readings amounting to a few hundred bytes on the wire, well under a kilobyte. The process posts it over HTTP to the fog node's ingest endpoint, so the input interface is an ordinary HTTP call the sensors make rather than a shared file or a direct write to storage.

The fog node is a Java service that serves its ingest endpoint on the runtime's own built-in HTTP server rather than a web framework, keeping the edge component dependency-light, and it runs as its own container on the edge host distinct

from the sensor processes and the cloud functions. It does real work before anything leaves the edge. Incoming readings arrive on the server's handler pool and are appended to the open window, which is a single map keyed by line and signal, while a scheduled task closes the window on a fixed timer; because those two happen on different threads, the map is guarded by one lock. The care is in how the boundary is handled: when the timer fires, the closing operation takes the lock only long enough to move the accumulated readings aside into a fresh snapshot and install an empty map for the next window, then releases the lock and does all the summarising and rule evaluation on the snapshot outside it. Holding the lock only for the swap, not for the reduction, means an arriving reading is never blocked behind a slow window close, and the next window begins accepting readings immediately.

When a window closes, each non-empty line-and-signal group is reduced to a summary carrying the count, minimum, maximum, average, and latest value, and that summary, not any raw reading, is evaluated against the alarm rules. The rules are held as small immutable records grouped by signal, each naming the summary field it tests, the comparison, the limit, and the alarm it raises, and a single evaluation routine reads the named field from the summary and fires the alarm when the comparison holds. This structure is what lets the rules mirror how the signals actually fail. Four of the five signals worsen only as they climb, so each carries one rule on its window average: vibration above its limit raises a bearing-wear risk, motor temperature above its limit raises an overheating alarm, acoustic level above its limit raises an anomaly, and power draw above its limit raises a power spike. Rotation speed is the exception and carries two rules, one on the window minimum and one on the maximum, because a machine is healthy only inside a speed band: dropping below the floor is an underspeed fault and climbing above the ceiling is an overspeed fault, and testing the window's extremes rather than its average is what catches a brief stall or overrun that an average would smooth away. The same rule set is grouped by signal and served verbatim on a thresholds endpoint, so the dashboard can disclose the limits without the disclosure ever driving evaluation.

The fog node's output interface is a real managed queue rather than a direct write to storage. On each window boundary it batches the closed summaries and sends them to Amazon SQS in grouped calls, each carrying up to the service's per-call limit of ten summaries, so a busy cycle costs a number of send operations proportional to the summaries closing divided by ten rather than one send per summary. At start-up it resolves the queue's address once, retrying while the queue is still being provisioned, because on a fresh deployment the queue may not exist when the edge host boots, and a failed send is logged and the cycle continues rather than crashing the node, a bias toward keeping the edge live that suits a monitoring workload where the next window is only seconds away.

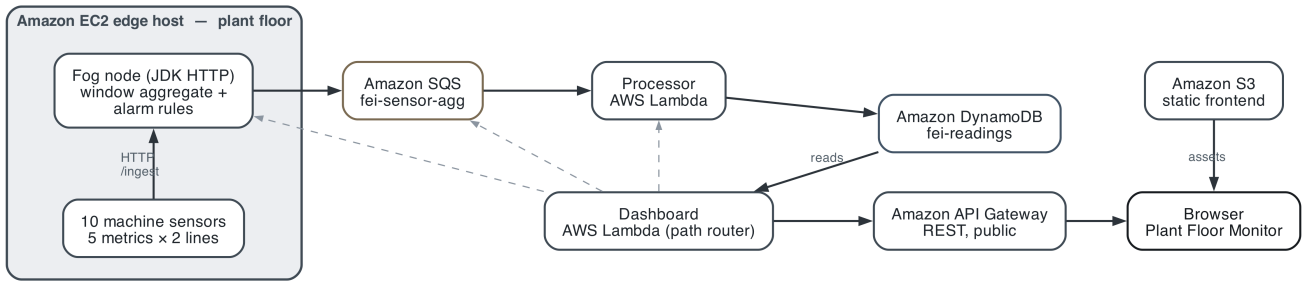


Fig. 1. Deployment topology. Ten sensor processes and the fog node share one Amazon EC2 edge host on the plant floor. Solid arrows are the ingest path, from the fog node through Amazon SQS and the processor function into the Amazon DynamoDB table, and the serving path, from the dashboard function's read of the stored windows out through Amazon API Gateway to the browser, with static assets from Amazon S3. The dashed edges are the three further status sources the dashboard polls for its health strip: the fog node's reachability, the queue's depth, and the processor function's deployment state.

B. Backend Layer

The cloud tier follows a single ingest path with a branch for the operator interface. A queue receives the fog node's summaries, one function drains the queue and writes to storage, and a second, separately deployed function reads that storage back to answer the dashboard. Splitting the write path from the read path is deliberate: ingestion and querying then scale and fail independently, so a surge of windows cannot slow the dashboard and a burst of operator activity cannot back up ingestion.

Queue instead of a direct call. The fog node could invoke the storage function directly and synchronously, dropping the queue and saving a hop. That option was declined. A synchronous call binds the edge to the availability and latency of the cloud function: throttle it, leave it cold, or take it down briefly, and the fog node stalls or sheds data as the back-pressure propagates toward the sensors. A queue severs that coupling, absorbs bursts, retries on the consumer's behalf, and lets the fog node treat a summary as delivered the moment it is enqueued [9]. On a ten-second cadence the small added latency is immaterial, while the decoupling and durability are exactly what an intermittent edge link needs.

Event-driven functions instead of a standing server. Both the ingestion and the interface tiers run as event-driven functions. This fits work that arrives in bursts and, for the interface, only intermittently: billing is per request rather than for idle capacity, and the platform adds and removes instances as demand shifts, with nothing reserved up front [8]. The recognised cost is the cold start after a period of idleness [10]; its effect here is small, because steady queue traffic keeps the ingestion function warm and the dashboard's own polling keeps the interface function warm. The processor function is triggered by the queue, holds no state between invocations, and is billed only for the work it does.

Key-value store instead of a relational database. A relational engine would offer joins and open-ended queries, neither of which this workload uses. Its access pattern is narrow and known in advance: append summaries in time order, then fetch the most recent windows for a given signal. Amazon DynamoDB, an on-demand key-value store built for that mix

of heavy writes and key-based reads, holds its performance as the table grows and spares the operational overhead of running a relational server, the balance the original Dynamo work set out and the managed service now carries [11], [12]. The key design follows straight from the query. Each stored window is keyed on its signal as the partition and on a sort value that leads with the window's closing timestamp and then the line identifier, so windows for one signal are already ordered chronologically inside a single partition and the newest are read with one bounded, descending, single-partition query rather than a scan; the line in the sort tail keeps the two lines' windows distinct within the same signal.

C. Serving Layer and Security Posture

The dashboard reaches the read function through a managed gateway that publishes a public REST endpoint. Object storage holds the static frontend, one page, a stylesheet, the page script, and a charting library kept in the repository rather than pulled from a content-delivery network. The read function keeps one set of query and health helpers that both a local server and the deployed function share; run locally it is served from the runtime's built-in HTTP server, and as a deployed function it is a small router that maps each gateway request path to the matching helper, since the local server's socket binding has no meaning inside a function. It exposes a small, read-only surface: a readings endpoint returning a recent series for one signal, a summary endpoint giving each line's latest window per signal, a thresholds endpoint that forwards the fog node's published rules, a backend-statistics endpoint, and a health endpoint assembled from four live sources, the store for the freshest window, the queue for its depth, the processor function for its deployment state, and the fog node for its reachability, returning a pipeline flag that is true only when a window actually reached the store recently. The backend-statistics endpoint reports the total number of stored windows, and because a count over the whole table can exceed a single response page, it follows the store's pagination contract and walks every page rather than trusting the first, so the figure stays correct as the table grows.

On the page, each production line is a column of five signal cards, and each card shows the signal's current value

per machine against a scaled meter, its alarm limit in plain terms, a short recent-history table, and a small trend line; a header reports how many signal types and machines are reporting, how many alarms are active, and how many records have been logged, and an alarm banner names any firing fault by line and signal. A fixed browser-side timer re-fetches the summary, health, and statistics every 2.5 seconds and repaints from whatever the responses hold, so the view stays current without a manual reload and without any server-push channel. The page learns which gateway origin to call from a single placeholder written into it and substituted with the real endpoint when the assets are uploaded; run locally, where one process serves both the page and the interface, the placeholder resolves to empty and requests fall back to the same origin. The security posture is modest and deliberate for a scaled demonstration: the fog node’s health and threshold endpoints answer without a credential check because they expose only aggregate window state and configured limits, never a live per-sensor reading, while the cloud tier relies on the managed services’ own access control and on scoping each function to the queue, table, and gateway it needs.

III. IMPLEMENTATION

The system is organised as four independent Java modules on the Java 17 runtime, one each for the sensor process, the fog node, the processor function, and the dashboard function, so that a change in one is built and tested on its own and the tiers never share mutable state through a common library. The fog node serves its ingest, health, and threshold endpoints on the runtime’s built-in HTTP server rather than a web framework, keeping the edge component dependency-light; JSON is handled with Jackson throughout, and the AWS SDK for Java carries the queue, table, and function calls. The sensor process and the fog node are packaged as containers and run on the edge host, while the processor and dashboard modules are built as function artefacts and deployed as event-driven functions; the frontend renders its trend with a locally vendored charting library so the page has no external dependency at load time. The edge tier is orchestrated with Docker Compose, the cloud tier is provisioned with Terraform, and continuous integration runs on GitHub Actions.

The alarm rules and the aggregation are kept in the fog module as plain data and pure functions, which is what makes the edge tier both testable and cheap to extend. A rule is an immutable record of a field, a comparison, a limit, and an alarm name, and the rules for all five signals sit in one table grouped by signal; adding a new fault, or giving an existing signal a second bound as rotation speed already has, is a new entry in that table rather than a new branch in a growing conditional, and because the evaluation reads the named field from an already-computed summary it never re-scans the raw readings. The aggregation is likewise a single pass over the window that yields the count, extremes, average, and latest value in one traversal, so the cost of closing a window is linear in the readings it holds and independent of how many rules will later consult the summary.

A. Testing and Continuous Integration

The system carries an automated suite of 46 tests, eighteen over the fog node, nineteen over the dashboard function, five over the processor function, and four over the sensor process. They run without any cloud connection by exercising the units against hand-written fake AWS clients, so each test can assert on the exact shape of the request a given call would issue rather than depending on a live service. The fog tests are the densest of the edge components, and deliberately so, because the edge is where the concurrency and the rules live: they cover the window aggregation arithmetic on known inputs, the batched relay’s chunking at the queue’s per-call limit, and each alarm rule at and around its limit. The alarm-rule tests are pointed at the boundaries, checking a value one step inside a limit and one step outside it, and rotation speed is exercised at both its floor and its ceiling so the two-sided band is pinned down rather than only one edge, since a rule engine that lets a signal carry a floor, a ceiling, or both is only trustworthy if both directions are actually tested. The dashboard tests are the densest overall, because the deployed function’s request routing is code the local server never runs and so is the part most exposed to a deployment-only mistake: they cover the per-line summary assembly from stored windows, the recent-series query that feeds the trend, the four health checks including the page-walking record count that must stay correct across a multi-page table, and the path routing the function performs against a gateway event. The processor tests cover the normalisation of a queued summary into a stored item, including the composite sort value that orders windows within a partition, and the sensor tests cover the bounded walk staying inside its band and the payload it encodes. The suite runs on every push through GitHub Actions, so a change that breaks a rule boundary, a batch size, a query shape, or the routing fails in continuous integration rather than after deployment.

Testing against fakes rather than a live queue was a deliberate choice with a cost worth acknowledging: a fake asserts on the request a call would make, not on how the managed service behaves under load, so the batched-send limit and the queue’s own retry semantics were reasoned about from the service documentation [13] and then confirmed against the live deployment rather than in unit tests.

B. Deployment and Live Verification

The cloud tier was provisioned to a real AWS account in a single Terraform apply that created twenty-four resources, with the account confirmed before anything was applied and the working state isolated so the apply could add only its own resources and never disturb anything already present. Describing the whole cloud tier as code, rather than clicking it together once, means the same definition builds the queue, both functions with their triggers and permissions [14], the table, the gateway with its single catch-all route [15], and the two buckets in one reproducible step, and tears them down as cleanly. The edge host runs the fog node and the ten sensor containers under Docker Compose; all the deployed resources carry a common prefix so the deployment is easy to identify

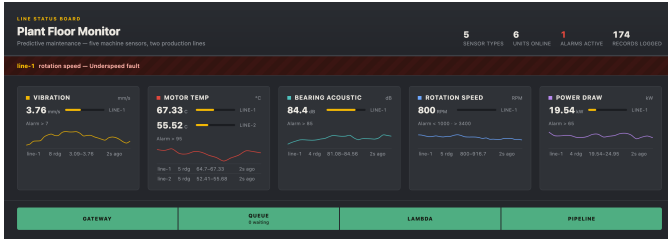


Fig. 2. The deployed plant-floor dashboard reading from the live endpoint. Each of the five machine signals shows its per-line current value against a scaled meter, its alarm limit, a recent-history table, and a trend line; the header reports signal types and machines reporting, active alarms, and records logged; the alarm banner names any firing fault by line; and the footer strip shows the four pipeline indicators.

and to remove as a unit, and a fixed public address in front of the edge host lets the read function reach the fog node’s health and threshold endpoints at the same location across restarts.

Verification was carried out against the live deployment rather than a local emulator, and Fig. 2 is the deployed dashboard during that check. Within about a minute of start-up the pipeline reported healthy end to end: the fog node, the queue, the processor function, and the freshest-window recency all read healthy, and the newest window was a few seconds old on each poll. Both lines rendered all five signals with their current value and recent range; the header counted the signal types and machines reporting and the records logged, which climbed across successive polls rather than staying static; and the alarm banner correctly named an underspeed fault on the line whose rotation speed had dipped below its floor, while leaving the healthy line unflagged, which is the two-sided speed rule doing exactly what a single-ended threshold could not. Reaching the dashboard from a browser also exercised the cross-origin path from the storage bucket to the gateway, which returned data cleanly.

IV. CONCLUSION

This project set out to build and, above all, to deploy a fog-to-cloud pipeline that turns five machine-condition signals across two production lines into an early warning of a failing machine, and it met that goal on a running system rather than on paper. The edge tier does genuine work before anything travels: it windows and reduces each line-and-signal stream and evaluates the alarm rules locally, so only compact summaries cross the network and a fault is named in the window it appears. The cloud tier scales through decoupling and elasticity rather than a pre-sized server, using a queue to absorb bursts and separate the write and read paths, event-driven functions billed only for the work they do, and an on-demand key-value store whose key design turns the dashboard’s one query into a single-partition read. The whole system was provisioned to a real account by one infrastructure-as-code apply and verified end to end against the live endpoint, with 46 automated tests guarding its behaviour and continuous integration running them on every change.

Two reflections stand out from building it. The first is that the alarm rules are only useful when they match the physics of failure, and rotation speed makes the point: four of the five signals degrade in one direction and are caught by a single limit on the window average, but a machine off its rated speed is faulty whether it has stalled or overrun, so it needs two rules testing the window’s extremes rather than one testing its middle, and building the rule engine so that a signal can carry a floor, a ceiling, or both was what let that distinction be expressed cleanly rather than bolted on. The second is that closing a window correctly under concurrency is the quiet centre of the edge tier: readings arrive continuously while a timer has to seal the window on a fixed boundary, and taking the lock only long enough to swap an empty buffer in, then reducing the sealed window outside the lock, is what keeps the hot ingest path from ever waiting on a slow summary.

The work has clear limits. It runs at a two-line scale with simulated sensors, its alarm limits are fixed rather than learned from a machine’s own baseline or adjustable at runtime, and its security posture is intentionally modest for a demonstration. Natural next steps are to widen it to many machines and real transducer feeds, to move the fixed limits to a managed configuration a maintenance team could tune per machine without a redeploy, to add authentication and a custom domain in front of the public endpoint, and to retain longer history so a signal’s slow drift toward its limit can be trended rather than only alarmed. None of these changes the shape of the pipeline, which is the point: the edge-aggregation and cloud-serving design carries from two lines to many without rethinking the tiers.

REFERENCES

- [1] J. Lee, B. Bagheri, and H.-A. Kao, “A Cyber-Physical Systems Architecture for Industry 4.0-based Manufacturing Systems,” *Manufacturing Letters*, vol. 3, pp. 18–23, 2015.
- [2] W. Zhang, D. Yang, and H. Wang, “Data-Driven Methods for Predictive Maintenance of Industrial Equipment: A Survey,” *IEEE Systems Journal*, vol. 13, no. 3, pp. 2213–2227, 2019.
- [3] L. Atzori, A. Iera, and G. Morabito, “The Internet of Things: A Survey,” *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [4] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, “Fog Computing and Its Role in the Internet of Things,” in *Proc. MCC Workshop on Mobile Cloud Computing*, 2012, pp. 13–16.
- [5] A. Yousefpour et al., “All One Needs to Know about Fog Computing and Related Edge Computing Paradigms: A Complete Survey,” *Journal of Systems Architecture*, vol. 98, pp. 289–330, 2019.
- [6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [7] M. Satyanarayanan, “The Emergence of Edge Computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [8] M. Armbrust et al., “A View of Cloud Computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [9] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [10] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, “Peeking Behind the Curtains of Serverless Platforms,” in *USENIX Annual Technical Conference*, 2018, pp. 133–146.
- [11] G. DeCandia et al., “Dynamo: Amazon’s Highly Available Key-value Store,” in *Proc. ACM SIGOPS Symp. Operating Systems Principles*, 2007, pp. 205–220.
- [12] M. Elhemali et al., “Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service,” in *USENIX Annual Technical Conference*, 2022, pp. 1037–1048.

- [13] Amazon Web Services, “Amazon Simple Queue Service Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/sqs/>
- [14] Amazon Web Services, “AWS Lambda Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/lambda/>
- [15] Amazon Web Services, “Amazon API Gateway Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/apigateway/>
- [16] IEEE, “Manuscript Templates for Conference Proceedings,” 2024. [Online]. Available: <https://www.ieee.org/conferences/publishing/templates.html>

The source code and deployment configuration are available at [GitHub repository URL].